

Securing Apache

In this part of the series, we focus on DoS/DDoS attacks, which have been among the major threats to Web servers since the beginning of the Web 2.0 era.

Denial of Service (DoS) attacks

The goal of a Denial of Service (DoS) attack is to disrupt some legitimate activity, such as browsing Web pages, e-mail functionality or the transfer of money from your bank account. It could even shut down the whole Web server. This denial-of-service effect is achieved by sending messages to the target machine such that the 'message' interferes with its operation and makes it hang, crash, reboot, or do useless work. In a majority of cases, the attacker's aim is to deprive clients of desired server functionality.

One way to interfere with legitimate operations is to exploit vulnerabilities on the target machine or application, by sending specially crafted requests targeting the given vulnerability (usually done with tools like Metasploit). Another way is to send a vast number of messages, which consume some key resource of the target machine, such as bandwidth, CPU time, memory, etc. The target application, machine, or network spends all of its critical resources on handling the attack traffic, and cannot attend to legitimate clients.

Of course, to generate such a vast number of requests, the attacker must possess a very powerful machine—with a sufficiently fast processor—and a lot of available network bandwidth. For the attack to be successful, it has to overload the target's resources. This means that an attacker's machine must be able to generate more traffic than a target, or its network infrastructure, can handle.

Attack scenario

Here is a simple scenario: an attacker sends a large number of requests to a Web server—for example, a website

that hosts HD image files at a particular URL, say `www.example.com/images/HD_images.html`. Let's also assume that this page contains about 50-60 images. Now, every time a user reloads this page, it consumes a large portion of the Web server's bandwidth. Now, here, an attacker could design a separate HTML page, with an *IFRAME* embedded in it, like what's shown below:

```
<html>
<iframe src=http://www.example.com/images/HD_images.html width=2
height=2>
</iframe>
</html>
```

Let's suppose that instead of a single *IFRAME*, the attacker copies and pastes the above *IFRAME* code 1000 times in the same page, and also adds a meta refresh tag as follows:

```
<html>
<head>
<meta http-equiv="refresh" content="2">
</head>
<iframe src=http://www.example.com/images/HD_images.html width=2
height=2>
</iframe>
<iframe src=http://www.example.com/images/HD_images.html width=2
height=2>
</iframe>
:
:
```

